MCAST
INSTITUTE OF INFORMATION &
COMMUNICATION TECHNOLOGIES

A Comparative Study of Monolithic and Modular Architectures

In a Dataset Driven Case Tracking Web Application

Mireya Francesca Vella

Bachelor of Science (Honours) in Creative Computing

MCAST
INSTITUTE OF INFORMATION &
COMMUNICATION TECHNOLOGIES

Hypothesis & Research Questions

Hypothesis

A modular or layered architecture is more maintainable and testable than a monolithic architecture.

RQ1

Does modular architecture reduce complexity vs monolithic?

RQ2

Does modular architecture improve test isolation and mock precision?

RQ3

Does architecture type affect ingestion throughput or API latency?

RQ4

Does architecture type affect how changes propagate through the codebase?

Literature Review

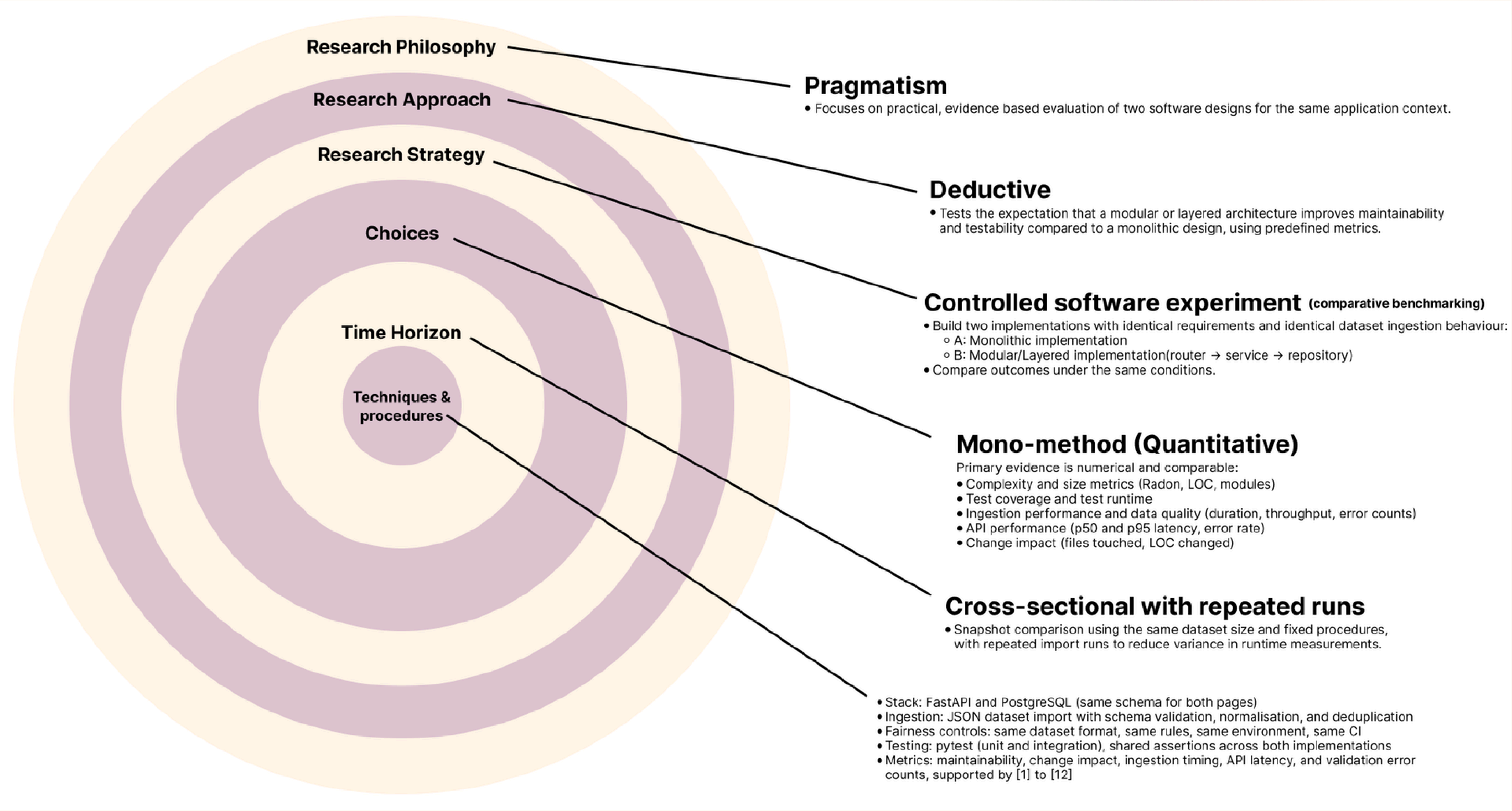
7 peer-reviewed sources across 3 interlocking themes

Architectural Smells & Maintainability	• [1] Mumtaz et al. (2021) Systematic mapping — smell detection landscape • [3] Sas et al. (2021) Co-change as a coupling and change-propagation proxy • [6] Esposito et al. (2025) Smell indicators correlated with static analysis warnings
Modularity Measurement & Boundary Quality	• [4] Pisch et al. (2024) M-score — empirically derived modularity metric • [5] Su & Li (2024) Modular monolith as an architectural trend
Remodularisation & Change Impact	• [2] Tan et al. (2023) REARRANGE — effort estimation for remodularisation • [7] Schroeder et al. (2021) Search-based remodularisation at Adyen (industrial)

MCAST
INSTITUTE OF INFORMATION &
COMMUNICATION TECHNOLOGIES

Research Design

Grounded in the Research Onion Framework



MCAST
INSTITUTE OF INFORMATION &
COMMUNICATION TECHNOLOGIES

Experiment Design & Fairness Controls

Architecture A: Monolithic

All routing, business logic & data access
in a single main.py (632 LOC)

VS

Architecture B: Modular

Router →Service → Repository
across 6 dedicated files (857 LOC)

Fairness Controls (identical across both implementations)

Same DB schema & migrations

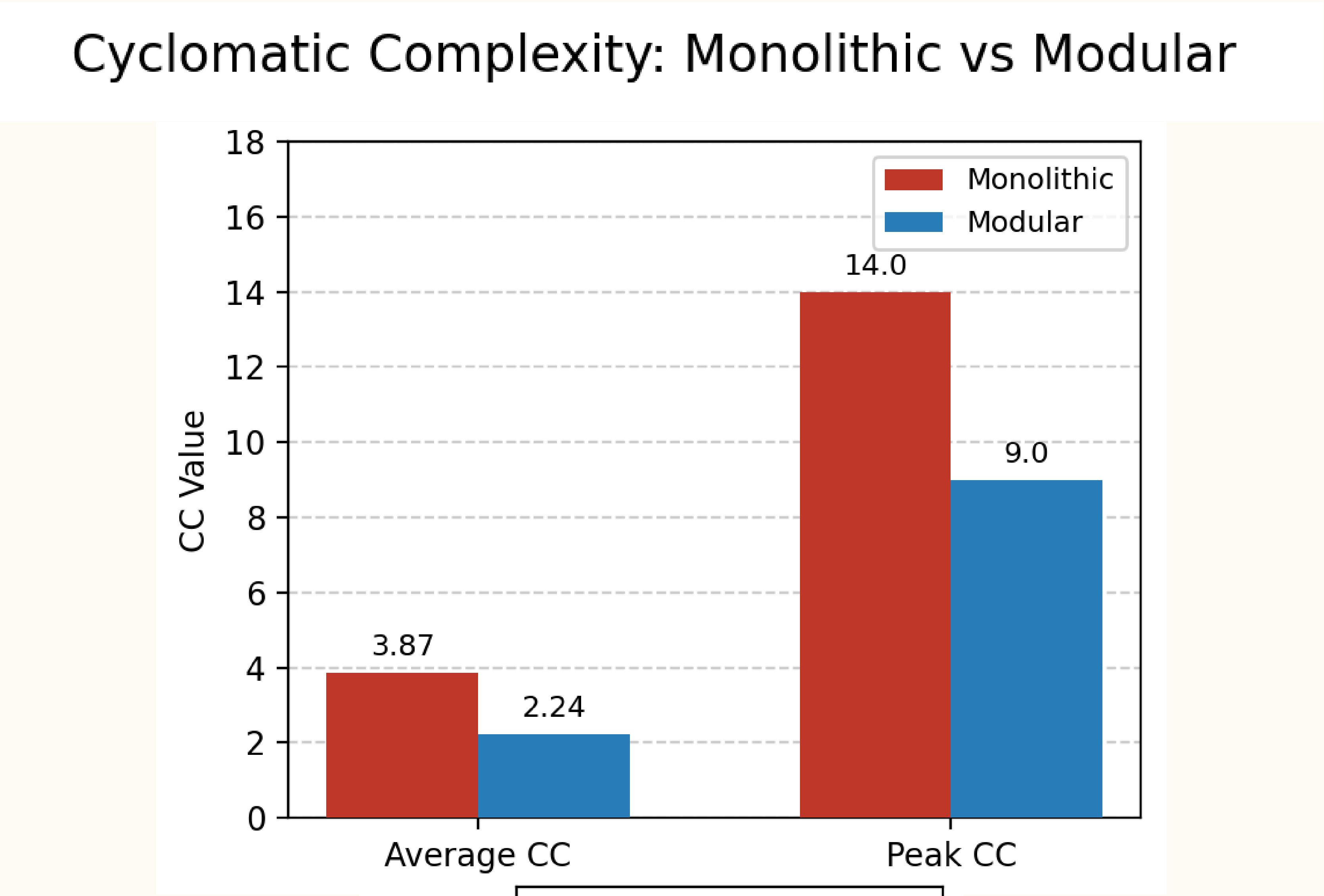
Same 5,000 record synthetic dataset

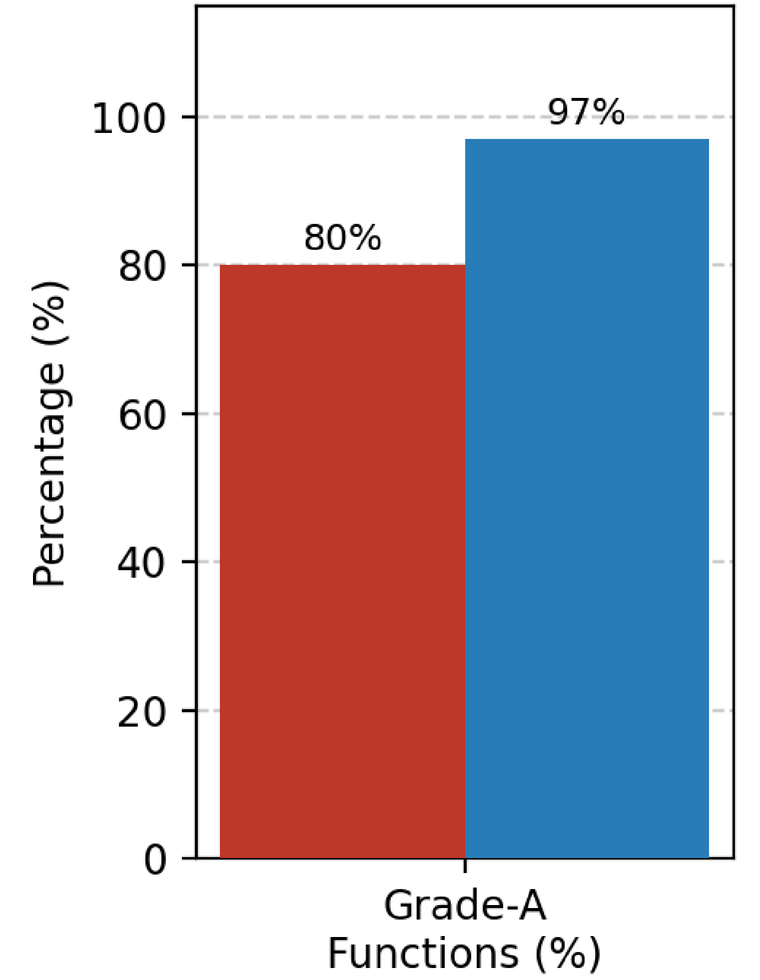
Same API contract & behaviour

Same 29-test pytest suite

Same CI environment

Dataset: Python script (generate_data.py) — 5,000 JSON records, 7 fields, fixed seed 42 for reproducibility. Used as sole input for both backends.





RQ1: Maintainability

LOC Distribution & Cyclomatic Complexity via Radon

Lines of Code

Monolithic: 632 LOC in 1 file

Modular: 857 LOC across 6 files (+35.6%)

Trade-off: explicit layer boundaries (Su & Li [5])

Lines of Code

Average CC: 3.87→ 2.24 (-42%)

Peak CC: 14 Grade C → 9 Grade B (-36%)

Grade-A functions: 80%→ 97% (+17 pp)

Lines of Code

Mono warning: line 379 of main.py (dense 632-line function)

Mod warning: services/cases.py:56
(layer obvious by filename)

RQ2: Testability

Shared 29-test API Contract Suite & Mock Injection Precision

Architecture A: Monolithic

29 / 29 tests passed
0 failures · 13 warnings
Runtime: 0.39 s

Architecture B: Modular

29 / 29 tests passed
0 failures · 13 warnings
Runtime: 0.53 s

Mock Injection Strategy

Concern	Monolithic target	Modular target (layer-boundary)
DB Connection	main.get_conn	routers.cases.get_conn
Bulk Insert	main.execute_values	repositories.cases.execute_values
Patch Accuracy	Module-level (broad)	Layer-boundary (precise)

Layer-boundary patching enables fault attribution to a specific architectural layer – a property monolithic design cannot offer without refactoring (Esposito et al. [6]).

RQ3: Performance

Proxy: pytest suite collection time (mocked DB)

Monolithic: 0.39s vsModular: 0.53s (+36%)

Attributable to 6 extra module imports, not logic

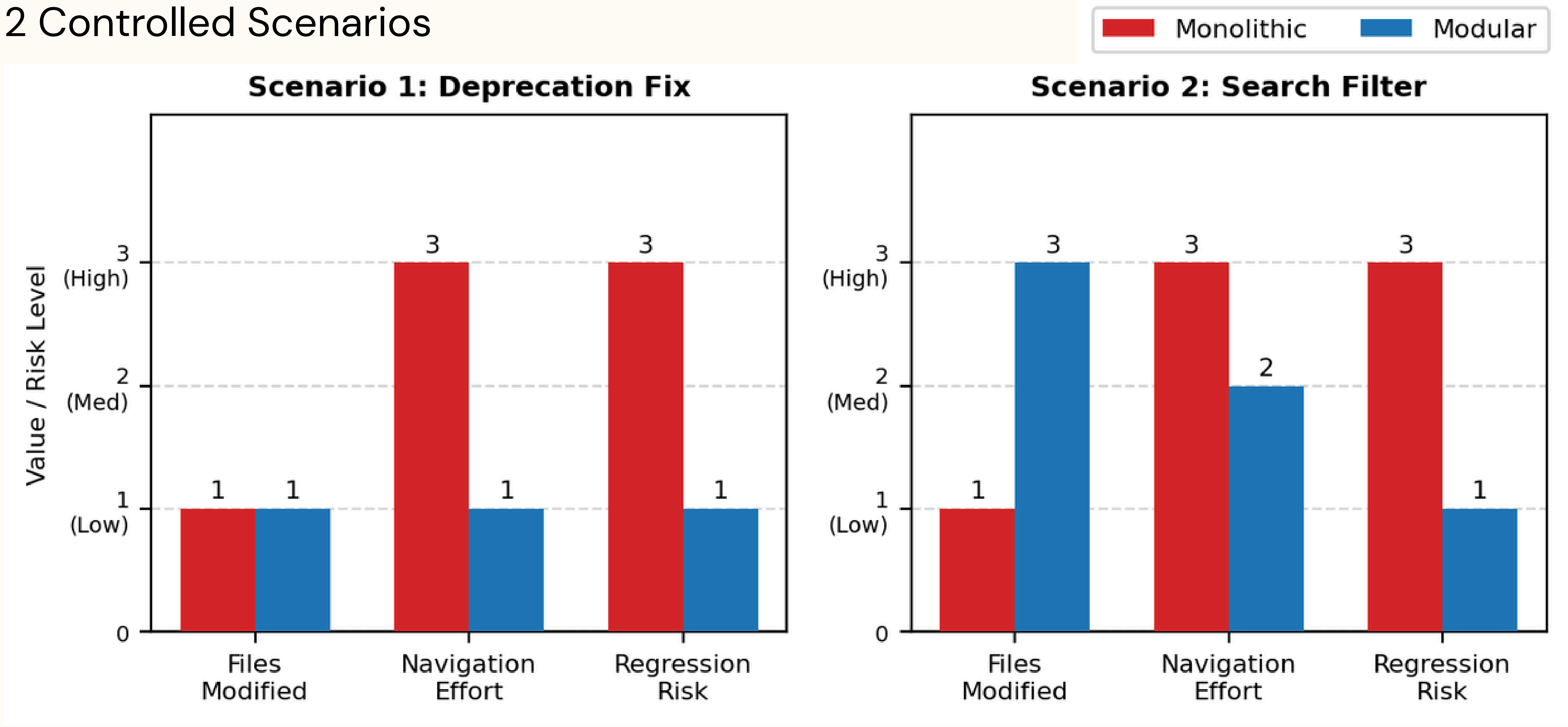
No structural performance penalty identified

Limitation

No live DB benchmark conducted (future work)

RQ4: Change Impact

2 Controlled Scenarios



Score Summary (1=Low, 2=Med, 3=High)

Dimension	S1 Mono	S1 Mod	S2 Mono	S2 Mod
Files Modified	1	1	1	3
Navigation Effort	3	1	3	2
Regression Risk	3	1	3	1

Total: Monolithic 14 vs Modular 9 – 36% lower change-impact score

Conclusion

Hypothesis confirmed across all four research questions

Limitations:

- *single stack (FastAPI/PostgreSQL)*
- *no live DB benchmark*
- *synthetic dataset only*

Future work:
live load test, additional stacks, remodularisation cost (Tan et al. [2])

Hypothesis confirmed:
The modular architecture is more maintainable and testable than the monolithic architecture.

RQ1 Maintainability
-42% avg CC | Peak CC 14→9 | Grade-A 80%→97%
Fault locatable by filename alone

RQ2 Testability
Layer-boundary mock injection vs module-level
Both suites: 29/29 passed - functional equivalence

RQ3 Performance
0.14 s proxy overhead (module imports, not logic)
No structural performance penalty identified

RQ4 Change Impact
Total score: 14 (Mono) vs 9 (Mod) - 36% lower
Boundaries constrain co-change scope & regression risk

Questions & Discussion

Thank you for listening!

Mireya Francesca Vella

Bachelor of Science (Honours) in Creative Computing